

MATLAB FDTD Simulation App Analysis

<https://gemini.google.com/app/26cd61090313f99e>

User prompt: now explain the algorithm of ftdt in simple but details

Response: The Finite-Difference Time-Domain (FDTD) algorithm calculates electromagnetic wave behavior step-by-step in time across a grid of space.

Step 1: Divide Space into Grid Points

Instead of solving continuous space, FDTD chops the domain into a discrete grid.

- **1D Mesh:** Space is divided into small 1D segments along a line of length N_z with spacing Δz .
- **2D Mesh:** Space is divided into a $N_x \times N_y$ grid of cells with spatial dimensions Δx and Δy .
- **Staggered Spatial Grid:** Electric fields (E) and Magnetic fields (H) sit half a grid step away from each other.

Step 2: Set the Time Step (The Speed Limit)

Time is broken into small time steps (Δt).

- **Courant Stability (CFL Condition):** The time step must be small enough that light cannot travel across a grid square faster than one time step:

$$\Delta t \leq \frac{1}{c_0 \sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}}}$$

- In your 2D code, this is calculated as $dt = 1 / (c_0 * \sqrt{(1/dx^2) + (1/dy^2)})$.

Step 3: Define Material Properties Across the Grid

Each grid point stores relative permittivity (ϵ_r) and relative permeability (μ_r).

- The code converts material properties into scaling factors for each cell:

$$m_{Ez} = \frac{c_0 \Delta t}{\epsilon_r}, \quad m_{Hx} = c_0 \Delta t, \quad m_{Hy} = c_0 \Delta t$$

- In your code, these are mapped using $mEz = (c_0 * dt) ./ ER$.

Step 4: The Time-Stepping Update Loop (Leap-Frog Scheme)

Inside the main loop, running from time step $T = 1$ to STEPS, the code alternates between updating magnetic and electric fields.

A. Update Magnetic Fields (H)

Using the current Electric field spatial differences, update H_x and H_y :

- **H_x Update Equation:**

$$H_x(x, y) \leftarrow H_x(x, y) - \frac{m_{Hx}}{\Delta y} [E_z(x, y + 1) - E_z(x, y)]$$

- **H_y Update Equation:**

$$H_y(x, y) \leftarrow H_y(x, y) + \frac{m_{Hy}}{\Delta x} [E_z(x+1, y) - E_z(x, y)]$$

B. Inject the Pulse Source

At time step T , introduce an incoming wave energy pulse into the grid:

- In 1D: $E_{\text{src}}(T) = \exp\left(-\left(\frac{t-t_0}{0.2\tau}\right)^2\right)$
- In 2D: Added to a line source at index $x = 20$ using $E_z(20, y) = E_z(20, y) + E_{\text{src}}(T)$.

C. Update Electric Fields (E)

Using spatial differences of the newly updated Magnetic fields, update E_z :

- E_z Update Equation:

$$E_z(x, y) \leftarrow E_z(x, y) + \frac{m_{Ez}(x, y)}{\Delta x} [H_y(x, y) - H_y(x-1, y)] - \frac{m_{Ez}(x, y)}{\Delta y} [H_x(x, y) - H_x(x, y-1)]$$

D. Apply Boundary Conditions

To prevent outward-moving waves from artificially reflecting off the grid boundaries back into the computational space:

- Save field values at neighboring inner nodes ($Ez_left = Ez(2, :)$) before updates.
- Apply First-Order **Mur Absorbing Boundary Conditions (ABC)** to overwrite outer edge values ($Ez(1, :) = Ez_left$).

Step 5: On-the-Fly Fourier Analysis (DFT)

To convert time-domain wave tracking into frequency spectra:

1. Continuous accumulation of complex phase factors:

$$\text{Ref}(f) = \sum E_{\text{ref}}(T) \cdot e^{-j2\pi f \Delta t \cdot T}$$

$$\text{Trn}(f) = \sum E_{\text{trn}}(T) \cdot e^{-j2\pi f \Delta t \cdot T}$$

2. Normalization against source energy:

$$\text{Reflectance} = \left| \frac{\text{Ref}(f)}{\text{Src}(f)} \right|^2, \quad \text{Transmittance} = \left| \frac{\text{Trn}(f)}{\text{Src}(f)} \right|^2$$

User prompt: What is spatial and temporal derivatives in this case, for this the project we are working on?

Response: In the FDTD method, **spatial derivatives** measure how an electromagnetic field changes across physical distance (space), while **temporal derivatives** measure how a field changes over time.

Because computers cannot calculate continuous calculus derivatives directly, FDTD replaces these derivatives with **central-difference approximations** using discrete grid steps in space ($\Delta x, \Delta y, \Delta z$) and time (Δt).

Spatial Derivatives ($\frac{\partial E}{\partial x}, \frac{\partial E}{\partial y}, \frac{\partial E}{\partial z}$)

What it means physically: It represents how much the Electric field E varies when moving from one spatial grid node to its neighboring node. According to Maxwell's equations, a spatially changing Electric field induces a Magnetic field over time.

How it appears in your project code:

- **1D Mode:** Measured across spatial step dz .
`(Ey(nz+1) - Ey(nz)) / dz` % Change in Ey along the z-axis[cite: 2]
- **2D Mode:** Measured across spatial steps dx and dy .
`(Ez(:, 2:Ny) - Ez(:, 1:Ny-1)) / dy` % Change in Ez along y-direction[cite: 2]
`(Ez(2:Nx, :) - Ez(1:Nx-1, :)) / dx` % Change in Ez along x-direction[cite: 2]

Temporal Derivatives ($\frac{\partial E}{\partial t}, \frac{\partial H}{\partial t}$)

What it means physically: It represents how much the field at a single, fixed point in space changes from the current time frame to the next frame (Δt).

How it appears in your project code: Rather than computing an explicit division by dt every time step, the temporal derivative is rearranged algebraically into the update scaling constants $m_{Ex}, m_{Ey}, m_{Hx}, m_{Hy}$:

- $m_{Ey} = (c0 * dt) ./ ER$
- $m_{Hx} = (c0 * dt) ./ UR$

Multiplying spatial field differences by these time-scaled constants steps the simulation forward in time (e.g., updating $H(t)$ to $H(t + \frac{1}{2}\Delta t)$ and $E(t)$ to $E(t + \Delta t)$).

Summary Comparison

Derivative	Physical Question	Code Implementation
Spatial Derivative	"How different is the field at point A compared to point B next to it?"	Spatial differences: <code>(Ez(2:Nx, :) - Ez(1:Nx-1, :)) / dx</code>
Temporal Derivative	"How does the field at point A change from frame 1 to frame 2?"	Time-stepping multipliers: <code>dt</code> embedded inside <code>mEz</code> and <code>mHx</code> updates

User prompt: explain the maxwells formula in simple temrs we are doing here

Response: In this project, Maxwell's equations describe how electric and magnetic fields interact and travel as electromagnetic waves.

In simple terms, four basic physical rules govern how electromagnetic fields behave, but two specific **curl equations** do the heavy lifting in your simulation:

The Two Core Rules Used in the Code

1. **Faraday’s Law (Changing Electric Fields Create Magnetic Fields)**
 - **In simple terms:** If an Electric Field (E) changes across space, it causes the Magnetic Field (H) to change over time.
 - **In the project:** The code takes spatial differences of the current electric field values ($Ey(nz+1) - Ey(nz)$) to calculate the next frame's magnetic field (Hx).

2. Ampère's Law (Changing Magnetic Fields Create Electric Fields)

- **In simple terms:** If a Magnetic Field (H) changes across space, it pushes and creates an Electric Field (E) over time.
- **In the project:** The code takes spatial differences of the updated magnetic fields ($H_x(nz) - H_x(nz-1)$) to calculate the next frame's electric field (E_y).

How Maxwell's Equations Are Simplified for the Simulator

In full 3D physics, Maxwell's equations require solving continuous multi-variable calculus vectors. To make them runnable on a computer, the program simplifies them through three steps:

1. Dimension Reduction

- **1D Mode:** The code assumes the wave moves along a single path (z -axis), meaning only one electric component (E_y) and one magnetic component (H_x) are needed.
- **2D Mode (TM_z):** The code tracks light spreading across a flat plane (x, y -grid) using one vertical Electric component (E_z) and two horizontal Magnetic components (H_x, H_y).

2. Material Scaling (ϵ_r and μ_r)

Maxwell's equations state that waves slow down or bounce when entering different materials. The code embeds permittivity (ϵ_r) and permeability (μ_r) into scaling multipliers (mEy , mEz , mHx):

- **Free space/Air:** $\epsilon_r = 1, \mu_r = 1$, allowing the wave to move at full speed (c_0).
- **Dense Medium (e.g., Water or Glass):** High ϵ_r scales down $mEz = (c_0 * dt) ./ ER$, slowing down the field update step and causing reflections at the material border.

3. Continuous Calculus → Computer Arithmetic

Computers cannot solve continuous derivatives ($\frac{\partial}{\partial t}$ or $\frac{\partial}{\partial z}$). The FDTD algorithm replaces smooth calculus derivatives with simple subtraction over small steps:

$$\text{Calculus Spatial Derivative } \left(\frac{\partial E_y}{\partial z} \right) \longrightarrow \text{Computer Array Subtraction } \left(\frac{E_y(nz + 1) - E_y(nz)}{\Delta z} \right)$$

By constantly recalculating these simple subtraction rules back and forth, the simulation transforms Maxwell's calculus equations into a moving animation of an electromagnetic wave.

User prompt: is central difference the best one here ? or other advanced differentiation techniques can be done?

Response: For standard FDTD implementations, the 2nd-order Central Difference scheme (Yee algorithm) is the industry standard due to its balance of simplicity, speed, and accuracy. However, depending on your simulation needs, other advanced differentiation techniques can be used.

Is Central Difference the Best Choice?

Yes, for standard FDTD applications. It provides **2nd-order spatial and temporal accuracy** $O(\Delta x^2, \Delta t^2)$ while requiring minimal computation memory and processing time per step. It maps naturally onto staggered grids, guaranteeing energy conservation and physical wave behavior without artificial amplitude damping.

Advanced Alternative Techniques

1. Higher-Order Finite-Difference Stencils (FDTD (2, 4) or (4, 4))

Instead of using just the immediate 2 neighboring grid points ($E_z(i+1) - E_z(i)$), higher-order finite differences use a wider stencil (e.g., 4 or 6 neighboring nodes):

- **Equation (4th-order spatial):**

$$\frac{\partial E_z}{\partial x} \approx \frac{-E(x+2) + 27E(x+1) - 27E(x-1) + E(x-2)}{24\Delta x}$$

- **Pros:** Drastically reduces **numerical dispersion** (waves of different frequencies traveling at incorrect speeds).
- **Cons:** Harder to handle material boundaries and complex geometries because wider spatial stencils cross boundary interface edges.

2. Pseudospectral Time-Domain Method (PSTD)

PSTD replaces spatial finite differences entirely by computing derivatives using the **Fast Fourier Transform (FFT)**:

$$\frac{\partial E_z}{\partial x} = \mathcal{F}^{-1} (jk_x \cdot \mathcal{F}\{E_z\})$$

- **Pros:** Achieves **infinite-order spatial accuracy** (spectral accuracy). You can use extremely coarse grids (as few as 2 grid points per wavelength instead of 10–50 points required by FDTD).
- **Cons:** High computational overhead per step due to repeated forward/inverse FFTs, and suffers from **Gibbs phenomenon** artifacts near sharp dielectric boundaries or metal interfaces.

3. Finite Element Time-Domain (FETD) / Discontinuous Galerkin (DGTD)

Instead of structured rectangular grids, these methods divide space into unstructured triangular or tetrahedral meshes.

- **Pros:** Excellent for highly complex, curved, or multi-scale geometrical objects without "stair-casing" errors.
- **Cons:** High mathematical complexity and computationally expensive matrix inversions.

Comparison Matrix

Method	Spatial Accuracy	Grid Requirement	Best For
Central Difference (Standard FDTD)	2nd-Order $O(\Delta x^2)$	10–20 points/ λ	General purpose, layered media, multi-material devices
Higher-Order Stencils	4th-Order $O(\Delta x^4)$	4–8 points/ λ	Long-distance wave propagation
Pseudospectral (PSTD)	Spectral (Infinite)	~2 points/ λ	Smooth media, large electrically-sized domains
Discontinuous Galerkin (DGTD)	Variable High-Order	Flexible Mesh	Complex, curved geometries and fine microstructures

User prompt: i mean forward difference formula, central diff formula of h2 h4 and richarsons extrapolation, then why central diff of h2?

Response: The choice to use the 2nd-order Central Difference scheme over Forward/Backward Differences, higher-order stencils ($\mathcal{O}(h^4)$), or Richardson Extrapolation comes down to the specific physical and computational requirements of the Yee FDTD algorithm.

1. Central Difference ($\mathcal{O}(h^2)$) vs. Forward/Backward Difference ($\mathcal{O}(h)$)

- **Order of Accuracy:**

- **Forward/Backward Difference:** Calculates derivative using one-sided points:

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \mathcal{O}(h)$$

This introduces a **1st-order truncation error** ($\mathcal{O}(h)$).

- **Central Difference:** Calculates derivative symmetrically:

$$\frac{f(x+h/2) - f(x-h/2)}{h} = f'(x) + \mathcal{O}(h^2)$$

By canceling out odd-order error terms, it achieves **2nd-order accuracy** ($\mathcal{O}(h^2)$) for the exact same computational effort.

- **Physical Energy Conservation:** 1st-order schemes (forward/backward) act like artificial dampening or amplification in time-stepping. They cause wave amplitudes to continuously decay or blow up. Central difference schemes conserve total electromagnetic energy over millions of timesteps.

2. Why Not 4th-Order Central Difference ($\mathcal{O}(h^4)$)?

A 4th-order spatial derivative uses a 4-point stencil:

$$\frac{\partial E_z}{\partial x} \approx \frac{-E(x+2h) + 27E(x+h) - 27E(x-h) + E(x-2h)}{24h} + \mathcal{O}(h^4)$$

While $\mathcal{O}(h^4)$ reduces numerical dispersion, it creates severe problems in FDTD:

- **Material Interfaces & Boundaries:** Electromagnetic simulations frequently involve sharp material boundaries (e.g., Air to Glass or Metal). A wide 4-point stencil crosses over material boundaries, mixing dielectric properties (ϵ_r) of two different materials together and generating non-physical boundary reflections.
- **Computational Cost:** It requires double the memory access and floating-point operations per time step.
- **Yee Staggered Grid Symmetry:** The spatial/temporal half-step offset ($\Delta x/2$, $\Delta t/2$) in Yee's staggered grid inherently aligns 2nd-order central differences directly at the calculation point without needing extra interpolation.

3. Why Not Richardson Extrapolation?

Richardson Extrapolation combines solutions from a coarse grid (h) and a fine grid ($h/2$) to eliminate lower-order error terms mathematically:

$$A_{\text{exact}} \approx A(h/2) + \frac{A(h/2) - A(h)}{3}$$

While useful for static differential equations, it is rarely used directly inside time-domain wave solvers like FDTD because:

- **Requires Multiple Simulation Runs:** To use Richardson extrapolation, you must run the entire time-stepping simulation twice (once on grid spacing h , and once on $h/2$). Running on $h/2$ quadruples the memory and computational time due to the CFL stability limit.

- **Time-Stepping Incompatibility:** In real-time wave propagation, field update values at timestep T depend directly on field values at timestep $T - 1/2$. You cannot easily extrapolate intermediate states on-the-fly inside a leap-frog update loop without saving massive multi-gigabyte spatial histories.

Summary: Why $\mathcal{O}(h^2)$ Central Difference Wins

Criteria	1st-Order (Forward/Backward)	2nd-Order Central ($\mathcal{O}(h^2)$)	4th-Order Central ($\mathcal{O}(h^4)$)	Richardson Extrapolation
Accuracy	Low ($\mathcal{O}(h)$)	Optimal ($\mathcal{O}(h^2)$)	High ($\mathcal{O}(h^4)$)	High ($\mathcal{O}(h^4)$)
Boundary Handling	Easy	Ideal (Compact 2-point)	Poor (Crosses interfaces)	Complex
Energy Stability	Unstable (Damped/Exploding)	Stable (Conserves Energy)	Stable	N/A
Compute / Memory	Very Low	Low & Fast	High	Extremely High (2x-8x runs)

User prompt: why there is 3 equation for 2d wave? why only one direction ffor electric field which is ez

Response: The reason your 2D simulation uses **3 field components** (E_z , H_x , and H_y)—with only **one electric field direction** (E_z)—comes down to how Maxwell's equations simplify in two dimensions.

1. In 3D, light has 6 field components

In full 3D space, Maxwell's equations track 6 field components:

- **Electric fields (E):** E_x, E_y, E_z
- **Magnetic fields (H):** H_x, H_y, H_z

2. In 2D, the 6 components split into two independent sets

When assuming the wave travels across a 2D flat plane (x, y -plane) and does not change along the z -axis (meaning $\frac{\partial}{\partial z} = 0$), Maxwell's curl equations break apart into **two completely independent 3-equation sets**:

Set A: Transverse Magnetic Mode (TM_z) — This is what your project uses!

- **Fields:** E_z, H_x, H_y
- **Equations (3 total):**
 1. $\frac{\partial H_x}{\partial t} \propto -\frac{\partial E_z}{\partial y}$
 2. $\frac{\partial H_y}{\partial t} \propto \frac{\partial E_z}{\partial x}$
 3. $\frac{\partial E_z}{\partial t} \propto \left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y} \right)$

Set B: Transverse Electric Mode (TE_z)

- **Fields:** H_z, E_x, E_y

- **Equations (3 total):**

$$1. \frac{\partial E_x}{\partial t} \propto \frac{\partial H_z}{\partial y}$$

$$2. \frac{\partial E_y}{\partial t} \propto -\frac{\partial H_z}{\partial x}$$

$$3. \frac{\partial H_z}{\partial t} \propto -\left(\frac{\partial E_y}{\partial x} - \frac{\partial E_x}{\partial y}\right)$$

Because Set A and Set B do not affect each other at all in 2D space, you only need to simulate **one set** to model full wave propagation physics.

Why only ONE Electric Field (E_z) and TWO Magnetic Fields (H_x, H_y)?

Imagine sending a ripple across a flat pool of water:

1. **The Vertical Direction (E_z):** Light points its electric field straight **UP and DOWN** out of the 2D plane (z -direction). It only needs one component (E_z) to represent the height/amplitude of the wave at any point (x, y) on the surface.
2. **The Horizontal Directions (H_x, H_y):** As that electric wave moves outward across the 2D plane, it pushes magnetic forces across the flat surface in both the **LEFT/RIGHT (x -axis)** and **FORWARD/BACKWARD (y -axis)** directions.

Therefore, you need H_x and H_y to guide the wave across the 2D surface, while E_z acts as the amplitude height pointing perpendicular to the plane.

How this maps directly to your MATLAB Code

In your 2D update loop, you can see these exact 3 equations implemented directly:

```
% 1. Update Hx using spatial changes of Ez along y-axis
Hx(:, 1:Ny-1) = Hx(:, 1:Ny-1) - (mHx / dy) * (Ez(:, 2:Ny) - Ez(:, 1:Ny-1));

% 2. Update Hy using spatial changes of Ez along x-axis
Hy(1:Nx-1, :) = Hy(1:Nx-1, :) + (mHy / dx) * (Ez(2:Nx, :) - Ez(1:Nx-1, :));

% 3. Update Ez using spatial changes of both Hx and Hy
Ez(2:Nx-1, 2:Ny-1) = Ez(2:Nx-1, 2:Ny-1) + ...
    (mEz(2:Nx-1, 2:Ny-1) ./ dx) .* (Hy(2:Nx-1, 2:Ny-1) - Hy(1:Nx-2, 2:Ny-1)) - ...
    (mEz(2:Nx-1, 2:Ny-1) ./ dy) .* (Hx(2:Nx-1, 2:Ny-1) - Hx(2:Nx-1, 1:Ny-2));
```[cite: 2]
```

*User prompt: but then how i can see this type of 3d flow? ez is blue which has only amplitude in the z axis where as red one is hx and hy maybe which is in the x-y plane*

Response: The reason you see this specific 3D plot—where  $E_z$  is a blue wave elevated in the vertical  $Z$ -axis and  $H_x$  is a red flat surface sitting near zero—comes down to a visualization trick used in MATLAB's plotting functions (mesh).

Here is how the plotting mechanics work under the hood:

## 1. How MATLAB Creates the 3D Heights

In MATLAB, when you pass a 2D grid matrix into `mesh(app.UIAxes, Matrix)`, MATLAB treats the **grid index coordinates** ( $x, y$ ) as the **flat horizontal floor** and uses the **matrix values as the height on the vertical  $Z$ -axis**.

- **The Blue Wave ( $E_z$ ):**

```
mesh(app.UIAxes, Ez, 'EdgeColor', 'b'); % Blue: Ez
```

The code passes  $E_z$  directly as the height values. Since the pulse energy is actively ringing in the electric field ( $E_z \approx -0.5$  to  $+0.5$ ), it pops out as a distinct **3D wave surface protruding up and down along the vertical axis**.

- **The Red Surface ( $H_x$ ):**

```
mesh(app.UIAxes, Hx, 'EdgeColor', 'r'); % Red: Hx
```

The magnetic field values  $H_x$  are also being plotted as vertical height. However, because  $H_x$  is at a different phase in the leap-frog cycle or scaled lower in magnitude at step 26, its values across most of the grid are close to 0. This causes it to render as a **flat red plane sitting right at  $Z = 0$**  (except for the localized spikes near the edges).

## 2. Is $H_x$ and $H_y$ physically in the $X$ - $Y$ plane?

**Yes, physically, but not on this specific plot:**

1. **The Physical Reality:** In  $\text{TM}_z$  physics,  $H_x$  points along the  $X$ -direction,  $H_y$  points along the  $Y$ -direction, and  $E_z$  points along the  $Z$ -direction.
2. **The Graph's Reality:** Because plotting real 3D vector arrows (vector quivers) across a  $120 \times 120$  grid is extremely laggy in real-time simulations, your code plots field magnitudes as vertical heights ( $Z$ -axis) rather than showing directional arrows on the floor.

## 3. Summary of Visual Components in your Screenshot

- **Blue Mesh Surface:**  $E_z$  magnitude (Electric field vertical height).
- **Red Mesh Surface:**  $H_x$  magnitude (Magnetic field component height mapped to the same vertical scale).
- **Green Cylinder:** The physical dielectric target (`FaceAlpha = 0.25` semi-transparent cylinder) placed at coordinates  $(x_{\text{center}}, y_{\text{center}}) = (80, 60)$ .
- **Camera Angle:** `view(app.UIAxes, [-37.5, 30])` tilts the axes into an isometric perspective so you can watch waves travel around the object in 3D space.

*User prompt: then is the output okay to my fdtd ? and if it is how do i explain that?*

**Response: Yes, your FDTD simulation output is functioning as expected.**

The snapshot demonstrates the fundamental mechanics of wave physics operating under the 2D  $\text{TM}_z$  formulation.

## How to Explain This Visual Output (Step-by-Step)

### 1. Explain the Visual Layers

- **The Blue Mesh ( $E_z$  Wave):** Represents the vertical Electric Field magnitude ( $E_z$ ). The wave is injected at the source boundary ( $x = 20$ ) and propagates forward as a smooth pulse.

- **The Green Cylinder (Obstacle):** Represents a physical dielectric object (a material with relative permittivity  $\epsilon_r > 1$ ) placed directly in the path of the wave at grid position  $(x, y) = (80, 60)$ .
- **The Red Plane ( $H_x$  Field):** Represents the horizontal Magnetic Field component in the  $x$ -direction. Because  $H_x$  is staggered half a step in space and time relative to  $E_z$ , its magnitude varies across the grid, spiking at wave gradients and sitting near zero elsewhere.

## 2. Explain the Physical Wave Interaction

- **Wave Delay & Refraction:** As the front edge of the blue  $E_z$  wave hits the green object, the higher permittivity ( $\epsilon_r$ ) inside the cylinder slows down the wave updates. This causes the wave phase inside the cylinder to lag behind the wave traveling through free space.
- **Scattering & Bouncing:** When the wave encounters the boundary between air ( $\epsilon_r = 1$ ) and the object, part of the wave energy reflects backward toward the source, while part transmits through the object.

## 3. Explain the 3D Height Rendering

- Clarify that the  $Z$ -axis height in the 3D plot represents **field intensity/amplitude**, not physical height in space.
- The code uses MATLAB's 3D mesh rendering (`mesh(Ez)`) viewed from an isometric camera angle (`view([-37.5, 30])`) to visually display how field values change across the 2D space.

User prompt: then where is the use of  $h_y$  ?

Response: Based on your code implementation for the 2D TM<sub>z</sub> FDTD simulation,  $H_y$  is actively used in the update equations to calculate the Electric Field ( $E_z$ ) and allow the wave to propagate sideways (along the X-axis).

Here is where and why  $H_y$  is used in your project:

### 1. Where $H_y$ Appears in Your Code

In your 2D update loop,  $H_y$  is calculated and used in two distinct steps:

#### Step A: Calculating $H_y$ from $E_z$ Spatial Differences

As the electric wave moves along the X-axis, spatial differences in  $E_z$  generate  $H_y$ :

```
% Update Magnetic Field Hy
Hy(1:Nx-1, :) = Hy(1:Nx-1, :) + (mHy / dx) * (Ez(2:Nx, :) - Ez(1:Nx-1, :));
```

(This measures the spatial derivative  $\frac{\partial E_z}{\partial x}$ ).

#### Step B: Updating $E_z$ using both $H_x$ and $H_y$

To push  $E_z$  forward in time, the code subtracts spatial variations of  $H_x$  along the Y-axis from spatial variations of  $H_y$  along the X-axis:

```
% Update Electric Field Ez using BOTH Hy and Hx
Ez(2:Nx-1, 2:Ny-1) = Ez(2:Nx-1, 2:Ny-1) + ...
 (mEz(2:Nx-1, 2:Ny-1) ./ dx) .* (Hy(2:Nx-1, 2:Ny-1) - Hy(1:Nx-2, 2:Ny-1)) - ...
 (mEz(2:Nx-1, 2:Ny-1) ./ dy) .* (Hx(2:Nx-1, 2:Ny-1) - Hx(2:Nx-1, 1:Ny-2));
```

(This calculates the 2D spatial curl  $\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial y}$ ).

## 2. Physical Meaning: Why $H_y$ is Essential for 2D Waves

Without  $H_y$ , your simulation would not be a true 2D simulation.

- $H_x$  controls Forward/Backward motion (Y-axis):  $H_x$  responds to  $E_z$  changing along the  $Y$ -direction.
- $H_y$  controls Left/Right motion (X-axis):  $H_y$  responds to  $E_z$  changing along the  $X$ -direction.
- **Together ( $H_x + H_y$ ):** They allow  $E_z$  to spread, scatter, and bend around objects (like the cylinder in your plot) in all 360° directions across the 2D surface.

## 3. Why Don't You "See" $H_y$ in the 3D Plot?

In your display code, you only see  $E_z$  (blue) and  $H_x$  (red) because the plotting command was specifically written to draw those two components:

```
mesh(app.UIAxes, Ez, 'EdgeColor', 'b'); % Plots Ez
mesh(app.UIAxes, Hx, 'EdgeColor', 'r'); % Plots Hx
```

$H_y$  is running behind the scenes inside the MATLAB workspace array  $H_y$ . It constantly feeds numbers into  $E_z$  every single timestep so that the blue wave can physically travel and scatter across the X-Y grid.

*User prompt: can i show this exact 3d type view using hx hy and ez as well? or the ez and hx combo is well enough?*

Response: The short answer is **yes, the current  $E_z$  and  $H_x$  combo is more than good enough for visualization**, but you **can** include  $H_y$  if you want to show the complete total magnetic field.

Here is a breakdown of why your current setup works, what adding  $H_y$  would look like, and how to choose between them:

### Why the $E_z$ and $H_x$ Combo is Well Enough

1. **Visual Clarity:** Plotting multiple 3D mesh surfaces on top of each other gets cluttered fast. Displaying one primary electric component ( $E_z$  in blue) and one primary magnetic component ( $H_x$  in red) keeps the graph clean and easy to read.
2. **Demonstrates Field Coupling:** Showing  $E_z$  alongside  $H_x$  successfully proves to an examiner or user that your simulation is actively running Maxwell's coupled equations (where  $E$  creates  $H$  and  $H$  creates  $E$ ).
3. **Standard Practice:** In electromagnetic wave physics, showing one electric component ( $E$ ) and one orthogonal magnetic component ( $H$ ) is the standard way to illustrate the complementary nature of electromagnetic fields.

### Can You Show $H_y$ as Well?

Yes! If you want to show all three field components in your 3D plot, you have two main options:

#### Option A: Add $H_y$ as a Third Mesh Surface (Green/Yellow)

You can simply add a third mesh line right after  $H_x$ :

```
mesh(app.UIAxes, Ez, 'EdgeColor', 'b'); hold(app.UIAxes, 'on'); % Blue: Ez[cite: 1]
mesh(app.UIAxes, Hx, 'EdgeColor', 'r'); % Red: Hx[cite: 1]
mesh(app.UIAxes, Hy, 'EdgeColor', 'g'); % Green: Hy
```

- **Pros:** Shows every single array calculated in your FDTD loop ( $E_z$ ,  $H_x$ , and  $H_y$ ).
- **Cons:** The plot will look quite crowded, as three overlapping wireframe meshes will fight for visual space alongside the 3D target cylinder.

### Option B: Combine $H_x$ and $H_y$ into Total Magnetic Field ( $H_{\text{total}}$ )

Instead of plotting  $H_x$  and  $H_y$  separately, you can calculate the total magnetic field magnitude using the Pythagorean theorem:

```
H_total = sqrt(Hx.^2 + Hy.^2); % Combine both magnetic components
mesh(app.UIAxes, Ez, 'EdgeColor', 'b'); hold(app.UIAxes, 'on'); % Blue: Ez[cite: 1]
mesh(app.UIAxes, H_total, 'EdgeColor', 'r'); % Red: Total H
```

- **Pros:** This is physically accurate because it represents the *complete* magnetic energy spreading across both the  $X$  and  $Y$  directions, while keeping the plot clean with only two mesh surfaces.

### Recommendation

- **For your presentation/report:** Stick with your **current  $E_z$  and  $H_x$  setup** or switch to the **Total Magnetic Field ( $H_{\text{total}}$ )**.
  - **How to explain it:** Tell your reviewer that  $E_z$  represents the electric wave amplitude, while the red mesh represents the accompanying magnetic field component pushing the wave through space. You can mention that  $H_y$  is actively calculated in the code background to handle the 2D spatial updates across the  $X$ -axis.
-